

[L] 阮崇轩 · 前端开发工作汇报 [R] 转转 [C]1

前端开发技术报告

校园二手物品交易平台——转转

阮崇轩 · 前端开发

华中农业大学 · 2026 年 7 月

目录

1 个人角色与工作概述	3
2 技术栈与工程架构	3
2.1 前端技术选型	3
2.2 项目目录结构	4
2.3 CSS 设计令牌体系	4
3 用户认证模块	5
3.1 登录页面	5
3.2 注册页面	5
3.3 找回密码页面	6
3.4 Axios 封装与 Token 自动刷新	6
4 消息通知系统	7
4.1 消息页面重写	7
4.2 通知铃铛组件	7
4.3 商品详情页联系卖家	8
5 订单详情页面	8
5.1 买家卖家双视角	8
5.2 发货物流录入	8
5.3 评价系统	8
6 UI 动效与交互优化	8
6.1 骨架屏加载	8
6.2 页面过渡动画	9
6.3 回到顶部按钮	9
6.4 导航栏滚动感知	9
6.5 按钮微交互	9
7 后端 Bug 修复贡献	9
7.1 验证码类型隔离（安全漏洞）	9
7.2 密码重置验证码提前消费（逻辑 Bug）	10
7.3 密码重置暴力破解防护	10
7.4 浏览量统计错误	10
7.5 订单通知联动	10
8 代码贡献统计	10

目录	2
9 核心技术亮点	11
10 消息系统实时通信原理	11
11 订单状态机设计	11
12 支付系统设计	12
13 密码重置安全设计	12
14 前端安全实践	13
15 项目中的技术决策与权衡	13
16 遇到的问题与解决方案	14

1 概述

我在项目中负责前端基础设施和用户交互模块的开发。具体来说：

- 前端项目从零搭建 (Vue3 + TypeScript + Vite)
- 登录、注册、找回密码页面
- 消息聊天系统 (站内信 + 通知)
- 订单详情页面 (支付、发货、收货、评价全流程)
- Axios 网络请求封装 (Token 自动管理)
- 后端 5 个 Bug 修复

以下按模块逐一说明做了什么、怎么做的、为什么这么做。

2 前端项目如何启动和运行

2.1 项目是怎么组织起来的

当你打开浏览器访问 `http://localhost:3000`，发生了什么：

1. Nginx 收到请求，返回 `index.html` (一个几乎空的 HTML 文件)
2. HTML 中引用了一个 `<script>` 标签，指向打包后的 JavaScript 文件
3. 浏览器下载并执行这个 JS 文件，Vue 框架启动
4. Vue Router 读取当前 URL，决定渲染哪个页面组件
5. 页面组件发出 API 请求获取数据，渲染到屏幕上

这叫做 单页应用 (SPA) —— 页面不会整体刷新，只替换变化的部分。

2.2 项目的文件结构

每个文件夹有明确的职责：

`src/`

- `api/index.ts` → 所有后端接口的封装 (30+ 个 API 函数)
- `router/index.ts` → URL 和页面的映射关系
- `stores/user.ts` → 登录状态管理 (全局共享)
- `utils/request.ts` → Axios 实例 + 拦截器 (请求前自动加 Token)

utils/auth.ts → localStorage 读写工具（存 Token）
views/ → 页面组件（每个 .vue 文件是一个页面）
layouts/ → 布局组件（导航栏+页脚公共结构）

3 Axios 封装：所有网络请求的基础

这是最重要也最容易被忽略的基础设施。项目中所有后端通信都经过这一个入口。

3.1 什么是 Axios

Axios 是一个 HTTP 客户端库。简单来说，它负责在浏览器和服务器之间发送和接收数据。比如你点击“登录”按钮，前端通过 Axios 把用户名密码发给后端，后端返回一个 Token，前端再把这个 Token 存起来。

3.2 为什么要封装

直接使用 Axios 的话，每次发请求都要手动写：

- 请求的基础 URL（/api/v1）
- 请求头中的 Token（Authorization: Bearer xxx）
- 错误处理（401 过期、403 无权、500 服务器错）

封装之后，这些逻辑只写一次，所有页面共享。

3.3 核心代码和解释

```
// 创建一个 Axios 实例，配置基础行为
const request = axios.create({
  baseURL: '/api/v1',           // 所有请求都从这个路径开始
  timeout: 15000,               // 超过 15 秒自动取消请求
  headers: { 'Content-Type': 'application/json' }
})

// 请求拦截器：每次发请求之前自动执行
request.interceptors.request.use(config => {
  const token = getToken()      // 从 localStorage 读取 Token
  if (token) {
    // 把 Token 放进请求头，格式是 "Bearer <token>"
    config.headers.Authorization = `Bearer ${token}`
  }
})
```

```
}  
  return config  
})
```

通俗理解：拦截器就像一个门卫，每次有人要出门（发请求），门卫自动检查有没有带钥匙（Token），有就帮你挂在身上，没有就空手出门。

3.4 Token 自动刷新：最精巧的部分

Token 有有效期（2 小时）。过期后需要拿 Refresh Token 去换新的。关键问题是：如果同时有 3 个请求都遇到 Token 过期，要不要刷新 3 次？当然不要。只需要刷新 1 次，刷新成功后 3 个请求都用新 Token 重试。

```
let isRefreshing = false // 标志位：是否正在刷新  
let failedQueue = []      // 等待队列  
  
// 当收到 401（未授权）时  
if (error.response?.status === 401) {  
  if (isRefreshing) {  
    // 如果已经在刷新了，把请求放进队列等待  
    return new Promise((resolve) => {  
      failedQueue.push({ resolve })  
    }).then(token => {  
      // 刷新成功后，用新 Token 重试  
      originalRequest.headers.Authorization = `Bearer ${token}`  
      return request(originalRequest)  
    })  
  }  
  isRefreshing = true  
  // 发起刷新请求...  
}
```

这个模式叫“请求队列排队”。类比：三个人同时发现门锁坏了（Token 过期），第一个去拿新钥匙，其余两个在旁边排队等着，新钥匙拿回来后三个人都用新钥匙进门。

4 登录与认证流程

4.1 完整登录流程

1. 用户在登录页输入用户名/邮箱/手机号 + 密码

2. 前端发送 POST /api/v1/user/login, 请求体包含 {account, password}
3. 后端验证密码 (用 BCrypt 比对哈希值), 成功则返回两个 Token:
 - Access Token: 2 小时有效期, 用于日常请求的身份验证
 - Refresh Token: 7 天有效期, 仅用于刷新 Access Token
4. 前端把两个 Token 存到 localStorage (浏览器的本地存储, 关闭页面也不会丢失)
5. 前端把用户信息 (昵称、头像、角色) 也存到 localStorage
6. 后续所有请求, 前端自动在请求头中携带 Access Token
7. 后端 JwtAuthenticationFilter 拦截每个请求, 解析 Token 获取用户 ID, 从数据库查用户状态, 设置认证上下文

4.2 为什么需要两个 Token

单 Token 的问题: 如果有效期短 (比如 30 分钟), 用户频繁操作时会不断掉线; 如果有效期长 (比如 30 天), Token 一旦泄露危害时间长。

双 Token 方案: Access Token 短 (2h) 但高频使用, Refresh Token 长 (7d) 但只在刷新时用。这样泄露 Access Token 的危害窗口只有 2 小时, 而用户体验不受影响 (自动刷新)。

4.3 登出时的安全处理

用户点击退出登录时, 不只是清除前端的 Token:

1. 前端发送 POST /api/v1/user/logout
2. 后端把当前 Token 的 ID 加入 Redis 黑名单, TTL 设置为 Token 的剩余有效时间
3. 后续请求中, JwtAuthenticationFilter 会检查 Token 是否在黑名单中
4. 即使在有效期内的 Token, 如果被加入了黑名单, 也会被拒绝

这解决了 Token 无法主动失效的问题 (JWT 是无状态的, 服务器无法在签发后撤销它, 除非用黑名单)。

5 注册与验证码机制

5.1 注册流程

1. 用户输入用户名、邮箱、密码
2. 点击”获取验证码”,前端发送 POST /api/v1/user/code,请求体 {target: email, type: "register"}
3. 后端生成 6 位随机数字,存到 Redis, Key 为 captcha:register: 邮箱, TTL 5 分钟
4. 后端通过 JavaMailSender 发送邮件到用户邮箱
5. 用户输入收到的验证码,点击注册
6. 前端发送 POST /api/v1/user/register,携带用户名、密码、邮箱、验证码
7. 后端从 Redis 读取验证码比对,通过后用 BCrypt 加密密码存储到 user 表
8. 注册成功后删除 Redis 中的验证码

5.2 验证码的安全设计

三个层面的安全措施:

1. **类型隔离**: Redis Key中包含类型前缀 (register/reset), 注册验证码不能用于重置密码, 反之亦然。这是因为我发现原始代码的验证码 Key 没有类型区分。
2. **频率限制**: 同一邮箱每 5 分钟最多发 3 次验证码, 超过返回 429 (请求过多)。防止恶意刷验证码。
3. **防消费**: 验证码只在业务流程真正成功后删除。如果用户输入了正确的验证码但密码不符合要求, 验证码不会被消耗, 用户可以修改密码后重试。

5.3 验证码防消费的实现细节

这是我从 Bug 修复中总结的经验。原始代码的顺序是:

1. 从 Redis 读取验证码 → 比对 ← 通过
2. 删除 Redis 中的验证码 ← 删了!
3. 从数据库查用户 ← 通过
4. 检查新旧密码是否相同 ← 失败! 抛异常

第 4 步失败后，验证码已经在第 2 步被删除了。用户重试时需要重新获取验证码，体验极差。

修复后的顺序：把第 2 步（删除验证码）移到整个方法的最后。只有当密码真正更新成功后才删除。这样中间任何一步失败，用户都可以直接重试而不需要重新获取验证码。

6 消息系统

6.1 数据流

消息系统的数据流是这样的：

1. 用户 A 在聊天输入框输入文字，点击发送
2. 前端调用 `POST /api/v1/message/send`，请求体 `{toUserId, content, type:"text"}`
3. 后端 `MessageServiceImpl.sendMessage` 插入一条记录到 `message` 表 (`fromUserId, toUserId, content, isRead=0`)
4. 同时创建一条 `Notification` 记录到 `notification` 表，内容是“XXX 给你发来一条消息”
5. 用户 B 的前端每 5 秒轮询一次 `GET /api/v1/message/conversation/{userId}`，发现新消息后更新界面

6.2 为什么用轮询而不是 WebSocket

WebSocket 是真正的“实时”通信（服务器主动推送），但需要额外的后端配置。对于这个项目，每 5 秒轮询一次的体验已经足够好——用户发消息后最多等 5 秒就能看到回复，对非即时通讯场景完全可接受。而且轮询方案更简单、更稳定、更容易调试。

6.3 轮询的具体实现

```
// 在组件挂载时启动 5 秒定时器
onMounted(() => {
  poller = setInterval(pollMessages, 5000)
})

// 在组件卸载时清除定时器（否则离开页面后还在请求！）
onUnmounted(() => {
  if (poller) clearInterval(poller)
```

```
})

async function pollMessages() {
  // 只有打开了某个会话时才轮询（避免无效请求）
  if (!selectedUserId.value) return

  const res = await messageApi.getConversation(selectedUserId.value, {page:1,size:50})
  const newMsgs = res.data || []

  // 如果消息数量变了，说明有新消息，更新界面
  if (newMsgs.length !== messages.value.length) {
    messages.value = [...newMsgs].reverse()
    loadConversations() // 同时刷新会话列表
  }
}
```

6.4 商品页联系卖家的实现

商品详情页有一个“联系卖家”按钮。点击后的流程：

1. 检查是否登录，未登录跳转登录页
2. 跳转到 `/message?to= 卖家 userId`
3. 消息页面读取 URL 参数 `route.query.to`，自动打开与该卖家的聊天窗口
4. 即使之前没聊过天（没有会话记录），也能直接开始新对话

这个设计的巧妙之处在于：URL参数携带了“我要和谁聊天”的信息，消息页面无需额外的 API 调用就能确定聊天对象。

6.5 通知系统的实现

头部导航栏的铃铛图标是一个 `el-popover` 组件（点击弹出浮动面板）。每 30 秒轮询 `GET /api/v1/notification/unread/count` 更新红点数字。点击铃铛时调用 `GET /api/v1/notification/list` 获取最近 10 条通知。

```
// 通知轮询（每30秒）
async function refreshUnread() {
  const nRes = await notificationApi.getUnreadCount()
  notifUnread.value = nRes.data?.count || 0
}
```

```
}  
// 点击通知后跳转到对应页面  
async function readNotif(n) {  
  await notificationApi.markRead(n.id)  
  if (n.type === 'order') router.push('/order/list')  
  else router.push('/message')  
}
```

通知的创建是由后端自动触发的。我在 `MessageServiceImpl` 和 `OrderServiceImpl` 中添加了通知创建逻辑：发消息 → 创建通知、下单 → 通知卖家、付款 → 通知卖家、发货 → 通知买家。

7 订单详情页面

7.1 角色判断：你怎么知道我是买家还是卖家

订单同时涉及两个人：买家（下单的人）和卖家（发布商品的人）。页面需要根据当前登录用户来判断应该显示什么按钮。

```
const currentUser = getUserInfo() // 从 localStorage 读取当前登录  
    用户  
  
// 判断是不是买家：当前用户的 ID 等于订单的 buyerId  
const isBuyer = computed(() => currentUser?.userId === order.value  
    ?.buyerId)  
  
// 判断是不是卖家：当前用户的 ID 等于订单的 sellerId  
const isSeller = computed(() => currentUser?.userId === order.value  
    ?.sellerId)
```

这两个计算属性会随着 `order.value` 的变化自动更新。Vue 的 `computed` 是响应式的——当它依赖的数据变化时，它自动重新计算。

7.2 根据状态和角色决定显示什么

```
<!-- 买家 + 待付款 = 显示 "去支付" 和 "取消订单" -->  
<div v-if="isBuyer && order.status === '待付款'">  
  <el-button @click="goToPay">去支付</el-button>  
  <el-button @click="cancel">取消订单</el-button>  
</div>
```

```
<!-- 卖家 + 待发货 = 显示 "录入物流发货" -->
<div v-if="isSeller_&&_order.status_===_ '待发货' ">
  <el-button @click="showShipDialog=_true">录入物流发货</el-button>
</div>

<!-- 买家 + 待收货 = 显示 "确认收货" -->
<div v-if="isBuyer_&&_order.status_===_ '待收货' ">
  <el-button @click="receive">确认收货</el-button>
</div>

<!-- 买家 + 已完成 + 还没评价 = 显示 "去评价" -->
<div v-if="isBuyer_&&_order.status_===_ '已完成' _&&_!reviewed">
  <el-button @click="showReviewDialog=_true">去评价</el-button>
</div>
```

Vue 的 `v-if` 指令根据条件决定是否渲染这段 HTML。如果条件不满足，这段 HTML 根本不会出现在页面上。

7.3 支付页面：模拟支付的四步流程

支付页面不是真正的支付（没有对接微信/支付宝），而是对支付流程的前端模拟。它的价值在于：让项目演示有完整的交易闭环。

1. **选择支付方式**：展示三种方式（微信、支付宝、校园卡），用不同颜色的卡片区分。用户选择后点击“确认支付”。
2. **处理中**：显示旋转动画，用 `setTimeout` 模拟 1.5-2.5 秒的支付处理时间。然后调用 `PUT /api/v1/order/{id}/pay`。
3. **成功**：SVG勾选动画 + 价格确认 + 2 秒倒计时自动跳转。
4. **失败**：抖动动画 + 错误信息 + 重试按钮。

状态管理使用一个 `step` 响应式变量（`'select' → 'processing' → 'success'/'error'`），不同步骤渲染不同 UI。

7.4 发货功能

卖家点击“录入物流发货”后，弹出对话框，选择物流公司（8 个选项）并输入快递单号。前端调用 `PUT /api/v1/order/{id}/ship`，请求体 `{logisticsCompany, logisticsNo}`。后端更新订单状态为“待收货”，记录发货时间，创建订单日志，通知买家。

8 后端 Bug 修复

8.1 验证码类型隔离（安全漏洞）

问题：原始代码中，验证码存储在 Redis 中的 Key 是 `captcha: 邮箱`，没有区分是注册验证码还是重置密码验证码。攻击者可以先注册一个账号（获取注册验证码），然后用这个验证码去重置别人的密码。

修复：将 Redis Key 改为 `captcha: 类型: 邮箱格式`。注册时 Key 为 `captcha:register:xxx@qq.com`。重置时 Key 为 `captcha:reset:xxx@qq.com`。两者互不可用。

8.2 密码重置暴力破解防护

问题：6 位数字验证码只有 100 万种可能，如果无限次尝试，理论上可以暴力破解。原始代码的 `resetPassword` 方法没有任何频率限制。

修复：同一邮箱每 15 分钟最多尝试 5 次。使用 Redis Key `pwd:reset:rate: 邮箱` 计数，超过限制返回 429。

8.3 浏览量统计错误

问题：卖家查看自己发布的商品也增加浏览量，导致数据不真实。

修复：在 `ProductServiceImpl.getProductDetail` 中增加判断——如果请求用户是该商品的发布者，则跳过浏览量 +1。

8.4 订单 VO 缺少字段

问题：后端返回给前端的订单数据中没有 `buyerId` 和 `sellerId`，前端无法判断当前用户是买家还是卖家。

修复：在 `OrderVO` 中增加 `buyerId` 和 `sellerId` 字段，在 `OrderServiceImpl` 中赋值。

8.5 订单状态变更自动通知

问题：订单状态变更后（付款、发货、收货），相关方（买家/卖家）没有任何通知，只能手动刷新页面查看。

修复：在 `OrderServiceImpl` 的各状态流转方法中增加通知创建逻辑。封装了 `notifyUser(userId, title, content)` 方法统一处理。

9 页面动效的实现

9.1 骨架屏：加载中的占位动画

当页面正在请求数据时，如果显示一片空白，用户会觉得卡住了。骨架屏预先画好几个灰色的卡片形状，提示用户“内容正在加载中”。

实现原理：用 CSS animation 做一个从左到右移动的光效（渐变色背景 + 移动 background-position）。数据加载完成后，v-if 切换到真实内容。

9.2 回到顶部按钮

监听 `window.scrollToY`，超过 400 像素时显示一个圆形按钮，点击后调用 `window.scrollTo({top: 0, behavior: 'smooth'})` ——浏览器原生平滑滚动。按钮的显示/隐藏用 CSS transition 做弹性缩放。

9.3 页面切换动画

Vue Router 的 `<router-view>` 支持 `<transition>` 包裹。设置 `mode="out-in"` 表示旧页面完全离开后新页面再进入，避免两个页面同时出现在屏幕上。动画效果用 CSS transition 实现——旧页面淡出，新页面从下方 16px 处淡入上滑。

10 总结

这个项目中我写的代码主要解决了以下问题：

- **前端如何与后端通信**：通过 Axios 封装，自动处理 Token 注入、过期刷新、错误分类
- **用户如何登录和保持登录状态**：JWT 双 Token 方案，Access Token 2h + Refresh Token 7d，自动刷新
- **验证码如何保证安全**：类型隔离（不同用途不通用）+ 频率限制（防暴力）+ 防消费（失败不删除）
- **消息如何实时更新**：前端轮询（每 5 秒拉一次），比 WebSocket 简单但效果足够
- **订单流转如何控制**：根据“当前用户角色 × 订单状态”两个维度决定显示什么操作按钮
- **支付流程如何模拟**：四步状态机（选择 → 处理中 → 成功/失败），纯前端实现